

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder
```

```
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
```

```
df = pd.read_csv(url)
```

```
print(df.head())
```

```

  PassengerId  Survived  Pclass  \
0            1         0        3
1            2         1        1
2            3         1        3
3            4         1        1
4            5         0        3

      Name     Sex  Age  SibSp  \
0  Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                Heikkinen, Miss. Laina  female  26.0      0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4    Allen, Mr. William Henry    male  35.0      0

   Parch  Ticket   Fare Cabin Embarked
0      0   A/5 21171   7.2500   NaN      S
1      0    PC 17599  71.2833   C85      C
2      0  STON/O2. 3101282   7.9250   NaN      S
3      0  113803   53.1000  C123      S
4      0  373450   8.0500   NaN      S
```

```
print(df.isnull().sum())
```

```

PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age          177
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin        687
Embarked       2
dtype: int64
```

```
df["Age"].fillna(df["Age"].mean(), inplace=True)
df["Embarked"].fillna(df["Embarked"].mode()[0], inplace=True)
```

/tmp/ipykernel\_286/3238960623.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through ch  
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]

```
df["Age"].fillna(df["Age"].mean(), inplace=True)
```

/tmp/ipykernel\_286/3238960623.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through ch  
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]

```
df["Embarked"].fillna(df["Embarked"].mode()[0], inplace=True)
```

```
print(df.describe())
```

```

  PassengerId  Survived  Pclass  Age  SibSp  \
count  891.000000  891.000000  891.000000  891.000000  891.000000
mean    446.000000    0.383838    2.308642    29.699118    0.523008
std    257.353842    0.486592    0.836071    13.002015    1.102743
min      1.000000    0.000000    1.000000     0.420000    0.000000
25%    223.500000    0.000000    2.000000    22.000000    0.000000
50%    446.000000    0.000000    3.000000    29.699118    0.000000
75%    668.500000    1.000000    3.000000    35.000000    1.000000
max    891.000000    1.000000    3.000000    80.000000    8.000000

   Parch  Fare
count  891.000000  891.000000
mean     0.381594   32.204208
std     0.806057   49.693429
min     0.000000    0.000000
25%     0.000000    7.910400
```

```
50%      0.000000  14.454200
75%      0.000000  31.000000
max       6.000000  512.329200
```

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          891 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     891 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
```

```
print(df.shape)
```

```
(891, 12)
```

```
print(df.dtypes)
```

```
PassengerId    int64
Survived        int64
Pclass          int64
Name            object
Sex             object
Age            float64
SibSp           int64
Parch           int64
Ticket          object
Fare            float64
Cabin           object
Embarked        object
dtype: object
```

```
df["Pclass"] = df["Pclass"].astype("category")
```

```
df["Age"] = df["Age"].astype(int)
```

```
df["Fare_normalized"] = (df["Fare"] - df["Fare"].min()) / (df["Fare"].max() - df["Fare"].min())
```

```
le = LabelEncoder()
```

```
df["Sex"] = le.fit_transform(df["Sex"])
df["Embarked"] = le.fit_transform(df["Embarked"])
```

```
df = pd.get_dummies(df, columns=["Embarked"])
```

```
print(df.head())
```

```
   PassengerId  Survived  Pclass  \
0             1         0       3
1             2         1       1
2             3         1       3
3             4         1       1
4             5         0       3

   Name      Sex  Age  SibSp  Parch  \
0  Braund, Mr. Owen Harris    1   22      1      0
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  0   38      1      0
2    Heikkinen, Miss. Laina    0   26      0      0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)    0   35      1      0
4    Allen, Mr. William Henry    1   35      0      0

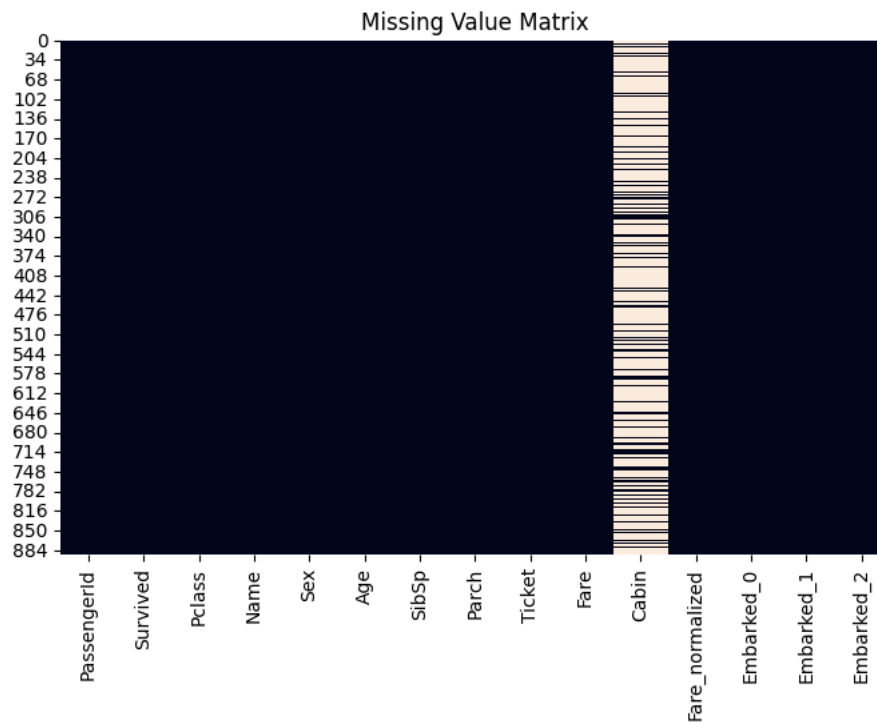
   Ticket    Fare  Cabin  Fare_normalized  Embarked_0  Embarked_1  \
0    A/5 21171  7.2500   NaN      0.014151      False      False
1    PC 17599  71.2833   C85      0.139136       True      False
2  STON/O2. 3101282  7.9250   NaN      0.015469      False      False
3   113803  53.1000  C123      0.103644      False      False
4   373450  8.0500   NaN      0.015713      False      False
```

```
Emarked_2
0      True
1     False
2      True
3      True
4      True
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

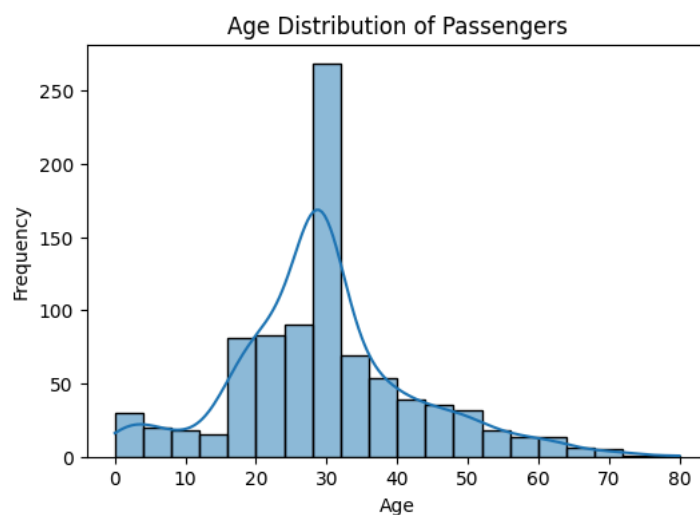
```
plt.figure(figsize=(8,5))
sns.heatmap(df.isnull(), cbar=False)

plt.title("Missing Value Matrix")
plt.show()
```



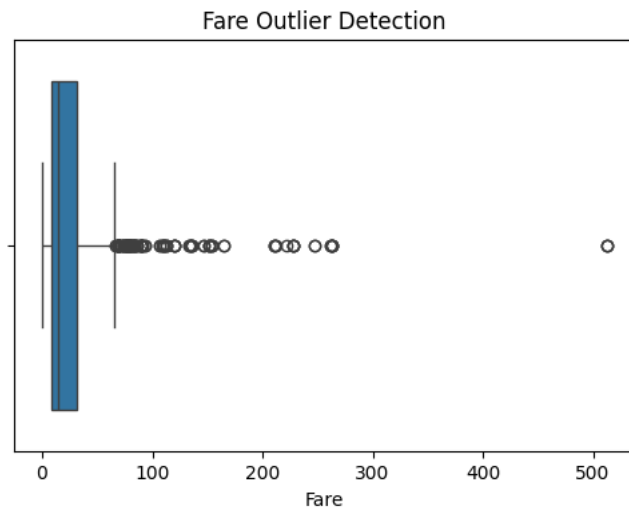
```
plt.figure(figsize=(6,4))
sns.histplot(df["Age"], bins=20, kde=True)

plt.title("Age Distribution of Passengers")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

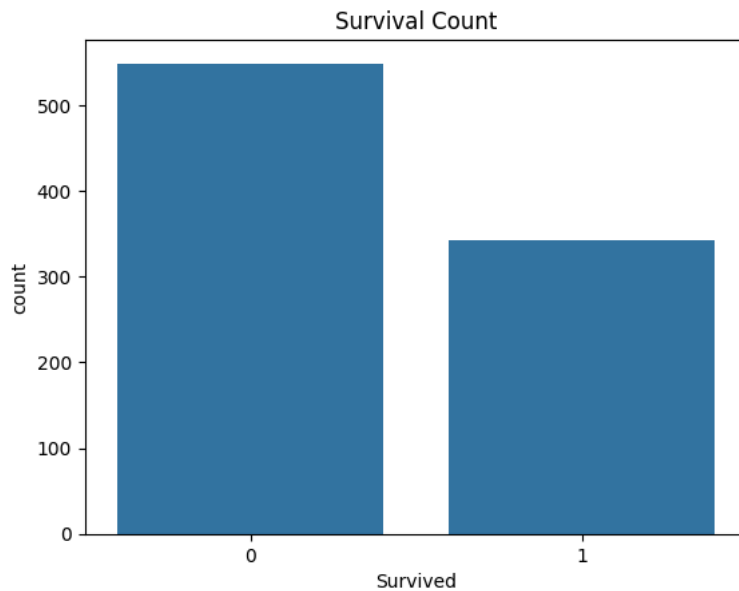


```
plt.figure(figsize=(6,4))
sns.boxplot(x=df["Fare"])
```

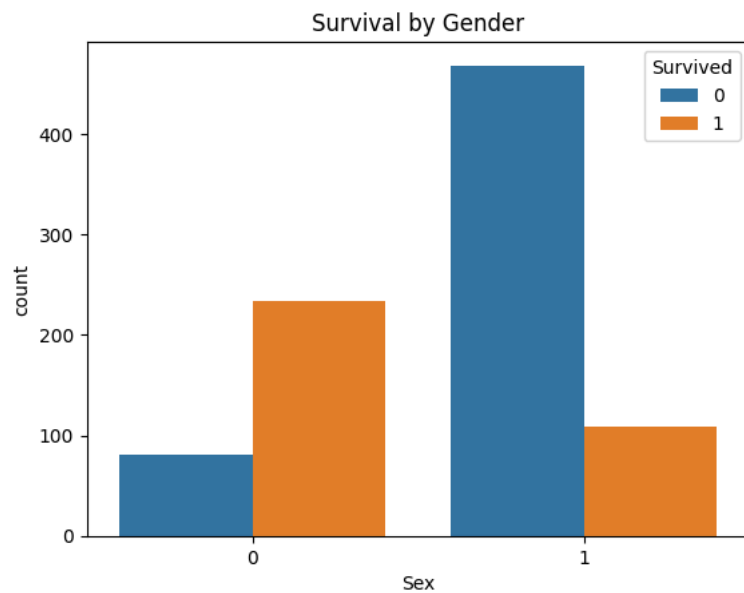
```
plt.title("Fare Outlier Detection")  
plt.show()
```



```
sns.countplot(x="Survived", data=df)  
  
plt.title("Survival Count")  
plt.show()
```

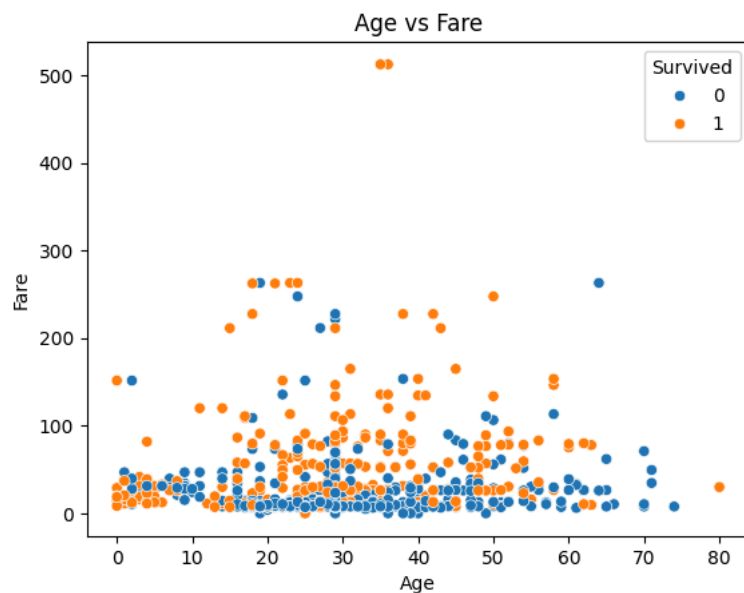


```
sns.countplot(x="Sex", hue="Survived", data=df)  
  
plt.title("Survival by Gender")  
plt.show()
```



```
sns.scatterplot(x="Age", y="Fare", hue="Survived", data=df)

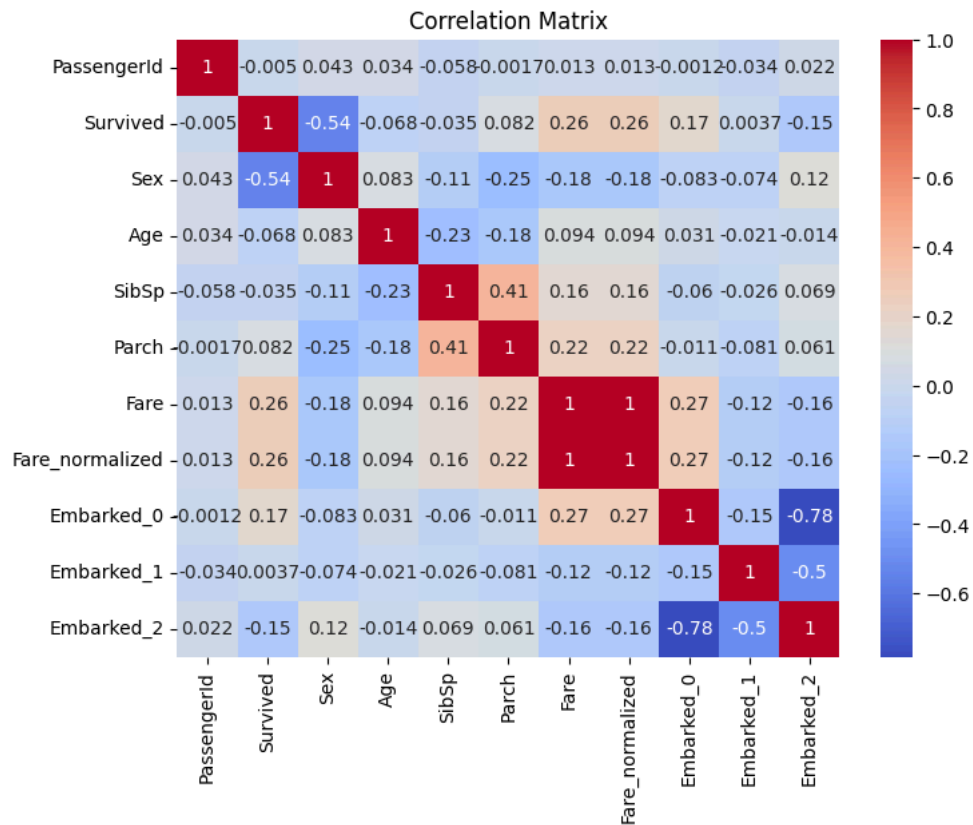
plt.title("Age vs Fare")
plt.show()
```



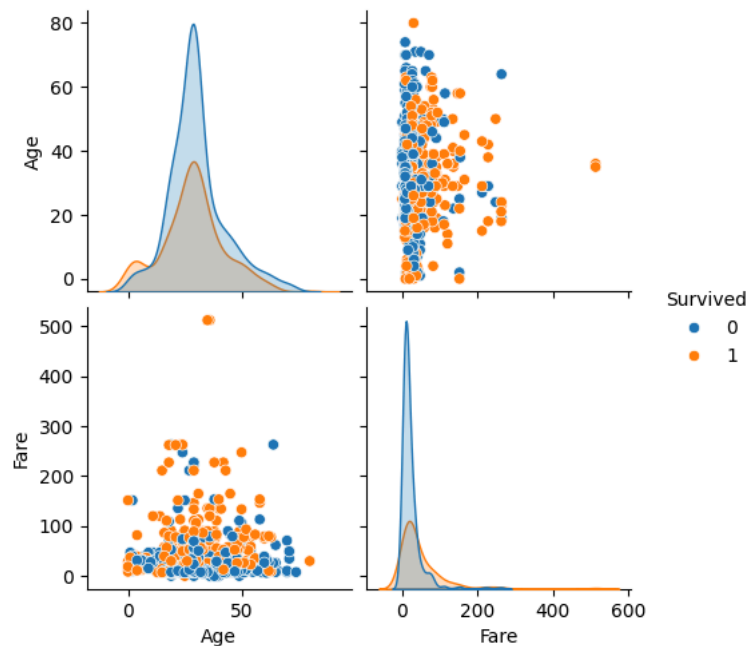
```
corr_matrix = df.corr(numeric_only=True)

plt.figure(figsize=(8,6))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm")

plt.title("Correlation Matrix")
plt.show()
```



```
sns.pairplot(df[["Age", "Fare", "Pclass", "Survived"]], hue="Survived")
plt.show()
```



```
sns.violinplot(x="Sex", y="Age", data=df)
plt.title("Age Distribution by Gender")
plt.show()
```

